Hidden Markov Models: Theory, Algorithms, and Applications in Data Science

Surya Rao Rayarao
Department of Statistics and Data Sciences
Department of Computer Science
The University of Texas at Austin
Austin, TX, USA
suryarao.r@utexas.edu

Naga Donikena
Department of Statistics and Data Sciences
Department of Computer Science
The University of Texas at Austin
Austin, TX, USA
naga.donikena@utexas.edu

Abstract—Hidden Markov Models (HMMs) constitute a fundamental class of probabilistic graphical models for sequential and temporal data, offering a principled framework for reasoning about systems whose underlying states are unobservable. Since their formalization by Baum and colleagues in the late 1960s and subsequent popularization through speech recognition research in the 1970s and 1980s, HMMs have become indispensable tools across a broad spectrum of data science disciplines. This survey provides a rigorous yet accessible treatment of HMM theory and practice, covering the mathematical foundations including the Markov property, emission distributions, and parameter spaces; the three canonical inference problems of evaluation, decoding, and learning; and the corresponding algorithmic solutions via the forward-backward algorithm, the Viterbi algorithm, and the Baum-Welch expectation-maximization procedure. We examine extensions including continuous-emission HMMs, higher-order models, and input-output variants. Real-world applications spanning speech recognition, bioinformatics, financial modeling, natural language processing, and anomaly detection are surveyed. Connections to related latent-variable models such as Kalman filters, conditional random fields, and deep sequential architectures are established. The survey aims to serve graduate students and practitioners in data science seeking a unified reference for both foundational theory and modern applied usage of hidden Markov models.

Index Terms—hidden Markov models, sequential data, latent structure, Viterbi algorithm, Baum-Welch, forward-backward algorithm, time series, probabilistic graphical models

I. Introduction

Many of the most consequential data streams encountered in science and engineering are fundamentally sequential: acoustic waveforms that encode spoken language, genomic sequences that encode protein function, financial price series that encode market dynamics, and network traffic logs that encode user behavior. A shared characteristic of such streams is temporal dependence—observations at one time point are statistically correlated with observations at neighboring time points—combined with latent structure: the observed signal is generated by an underlying process whose state is not directly accessible to the analyst.

Hidden Markov Models address both challenges simultaneously. An HMM posits a discrete latent state sequence that evolves as a Markov chain, coupled with an observation model that generates each measurement conditionally on the current hidden state. The model is expressive enough to capture rich temporal patterns yet structured enough to admit exact polynomial-time inference algorithms.

The origins of HMMs are credited to Baum, Petrie, Soules, and Weiss [1], who established the expectation-maximization procedure for parameter estimation. The decisive breakthrough in large-scale applicability came through their adoption in automatic speech recognition by Baker [2] and Jelinek and colleagues at IBM [3], a tradition later continued at AT&T Bell Laboratories and Carnegie Mellon University [4]. Rabiner's 1989 tutorial [4] remains the most widely cited exposition of the classical theory. The subsequent decades witnessed the adoption of HMMs in computational biology [8], financial econometrics [7], gesture and activity recognition [9], and many other fields.

Within the course framework of Data Science: Advanced Predictive Models, HMMs are simultaneously an example of (1) principled dependency modeling in structured data, (2) a fully specified generative model for dependent sequences, (3) an engine for probabilistic forecasting that explicitly accounts for temporal structure, and (4) an unsupervised method for discovering latent categorical structure in complex observational data. Each of these themes is developed in turn throughout this survey.

A. Paper Organization

Section II establishes notation and mathematical prerequisites. Section III develops the formal HMM specification and its probabilistic properties. Section IV presents the three canonical algorithms. Section V provides worked numerical examples. Section VI surveys real-world applications in data science. Section VII discusses limitations and modeling assumptions. Section VIII situates HMMs within the landscape of related models. Section IX concludes.

II. Mathematical Preliminaries

A. Notation

Throughout this paper we use the following conventions. Scalars are denoted by lowercase italic letters (a, b, π). Vectors are boldface lowercase $(\boldsymbol{\pi}, \mathbf{b})$. Matrices are boldface uppercase $(\mathbf{A}, \mathbf{B})$. Random variables are uppercase (X, Y, S). The probability of an event E is written $P(E)$; probability density or mass functions are written $p(\cdot)$ with the argument identifying the variable. The notation $[N] = \{1, 2, \ldots, N\}$ denotes the integer index set. We write $\mathbb{R}$ for the reals and $\mathbb{R}_{>0}$ for strictly positive reals. The indicator function is $\mathbf{1}[\cdot]$.

For a sequence of random variables indexed by discrete time $t = 1, \ldots, T$ we write $X_{1:T} = (X_1, X_2, \ldots, X_T)$. The notation $x_{1:T}$ denotes a realized sequence.

B. The Markov Property

Definition 1 (First-Order Markov Chain). A sequence of random variables $(S_1, S_2, \ldots, S_T)$ taking values in a finite state space $\mathcal{S} = \{1, 2, \ldots, N\}$ is a first-order Markov chain if for all $t \geq 2$ and all state sequences $s_1, \ldots, s_t \in \mathcal{S}$,

$$P(S_t = s_t \mid S_{t-1} = s_{t-1}, \ldots, S_1 = s_1) = P(S_t = s_t \mid S_{t-1} = s_{t-1}). \tag{1}$$

The Markov property formalizes the notion that the present state captures all information relevant for predicting the future; the past is conditionally independent of the future given the present. A time-homogeneous Markov chain is fully characterized by an initial distribution $\boldsymbol{\pi} \in \Delta^{N-1}$ and a transition matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, where Δ^{N-1} is the $(N-1)$-dimensional probability simplex.

C. Discrete Probability Distributions

For discrete emission alphabets $\mathcal{V} = \{v_1, \ldots, v_M\}$, the multinomial distribution Categorical($\mathbf{b}$) with parameter vector $\mathbf{b} = (b_1, \ldots, b_M)$, $b_k \geq 0$, $\sum_k b_k = 1$, assigns probability b_k to symbol v_k. For continuous emissions, common choices include the Gaussian $\mathcal{N}(\mu, \sigma^2)$ and Gaussian mixtures.

D. Expectation-Maximization

The Expectation-Maximization (EM) algorithm [6] is a general iterative method for maximum likelihood estimation in models with latent variables. Let $\boldsymbol{\theta}$ denote model parameters, $\mathbf{O}$ observed data, and $\mathbf{S}$ latent variables. EM alternates between:

- E-step: Compute $Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(k)}) = \mathbb{E}_{p(\mathbf{S} \mid \mathbf{O}, \boldsymbol{\theta}^{(k)})}[\log p(\mathbf{O}, \mathbf{S} \mid \boldsymbol{\theta})]$.
- M-step: Set $\boldsymbol{\theta}^{(k+1)} = \arg\max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^{(k)})$.

EM guarantees monotone non-decrease of the observed-data log-likelihood $\log p(\mathbf{O} \mid \boldsymbol{\theta})$ [6].

III. Core Theory of Hidden Markov Models

A. Formal Definition

Definition 2 (Hidden Markov Model). A Hidden Markov Model is a tuple $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$ where:

1) $\boldsymbol{\pi} = (\pi_i)_{i=1}^N$ is the initial state distribution, $\pi_i = P(S_1 = i)$.
2) $\mathbf{A} = (a_{ij})_{N \times N}$ is the transition probability matrix, $a_{ij} = P(S_{t+1} = j \mid S_t = i)$, with $a_{ij} \geq 0$ and $\sum_{j=1}^N a_{ij} = 1$ for all i.
3) $\mathbf{B} = (b_j(k))_{N \times M}$ is the emission probability matrix for a discrete observation alphabet, $b_j(k) = P(O_t = v_k \mid S_t = j)$, with $b_j(k) \geq 0$ and $\sum_{k=1}^M b_j(k) = 1$ for all j.

The joint distribution of a state sequence $\mathbf{s} = s_{1:T}$ and observation sequence $\mathbf{o} = o_{1:T}$ factors as:

$$P(\mathbf{s}, \mathbf{o} \mid \lambda) = \pi_{s_1} \prod_{t=1}^{T-1} a_{s_t s_{t+1}} \cdot \prod_{t=1}^{T} b_{s_t}(o_t). \tag{2}$$

The key structural assumption is conditional independence of observations: given the latent state sequence, observations are mutually independent,

$$P(O_t \mid S_{1:T}, O_{1:t-1}, O_{t+1:T}) = P(O_t \mid S_t). \tag{3}$$

This assumption, together with the Markov property on states, defines the HMM as a directed graphical model with the factorization in (2). The graphical structure is shown schematically: states form a chain $S_1 \to S_2 \to \cdots \to S_T$ and each O_t depends only on S_t.

B. The Three Problems of HMMs

Rabiner [4] identified three fundamental computational problems that must be solved for an HMM to be useful in practice:

1) Evaluation (Likelihood). Given model λ and observations $\mathbf{o}_{1:T}$, compute $P(\mathbf{o}_{1:T} \mid \lambda)$.
2) Decoding (State Inference). Given λ and $\mathbf{o}_{1:T}$, find the most probable hidden state sequence $\mathbf{s}^* = \arg\max_{\mathbf{s}} P(\mathbf{s} \mid \mathbf{o}_{1:T}, \lambda)$.
3) Learning (Parameter Estimation). Given observations $\mathbf{o}_{1:T}$ and model structure (number of states N, alphabet size M), find $\lambda^* = \arg\max_{\lambda} P(\mathbf{o}_{1:T} \mid \lambda)$.

C. Probability of Observations: The Likelihood

Directly computing $P(\mathbf{o}_{1:T} \mid \lambda) = \sum_{\mathbf{s}} P(\mathbf{s}, \mathbf{o}_{1:T} \mid \lambda)$ by exhaustive summation requires $O(N^T)$ operations and is computationally infeasible. The forward algorithm resolves this via dynamic programming, reducing the complexity to $O(N^2 T)$.

Definition 3 (Forward Variable).

$$\alpha_t(i) = P(O_1 = o_1, \ldots, O_t = o_t, S_t = i \mid \lambda). \tag{4}$$

The forward variables satisfy the recursion:

$$\alpha_1(i) = \pi_i \, b_i(o_1), \quad i \in [N], \tag{5}$$

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) \, a_{ij} \right] b_j(o_{t+1}), \quad t = 1, \ldots, T-1. \tag{6}$$

The likelihood is then:

$$P(\mathbf{o}_{1:T} \mid \lambda) = \sum_{i=1}^{N} \alpha_T(i). \tag{7}$$

Definition 4 (Backward Variable).

$$\beta_t(i) = P(O_{t+1} = o_{t+1}, \ldots, O_T = o_T \mid S_t = i, \lambda). \tag{8}$$

The backward variables satisfy:

$$\beta_T(i) = 1, \quad i \in [N], \tag{9}$$

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} \, b_j(o_{t+1}) \, \beta_{t+1}(j), \quad t = T-1, \ldots, 1. \tag{10}$$

D. Posterior State Distributions

Using both forward and backward variables, one can compute the posterior state marginals needed for parameter re-estimation:

$$\gamma_t(i) = P(S_t = i \mid \mathbf{o}_{1:T}, \lambda) = \frac{\alpha_t(i)\beta_t(i)}{\sum_{j=1}^{N} \alpha_t(j)\beta_t(j)}, \tag{11}$$

and the pairwise posterior:

$$\xi_t(i,j) = P(S_t = i, S_{t+1} = j \mid \mathbf{o}_{1:T}, \lambda) = \frac{\alpha_t(i) \, a_{ij} \, b_j(o_{t+1}) \, \beta_{t+1}(j)}{\sum_{i'} \sum_{j'} \alpha_t(i') \, a_{i'j'} \, b_{j'}(o_{t+1}) \, \beta_{t+1}(j')} \tag{12}$$

These quantities have direct interpretations as expected sufficient statistics, which are central to the Baum-Welch learning algorithm.

E. Continuous-Emission HMMs

When observations are continuous, the emission probabilities $b_j(o_t)$ are replaced by probability density functions. A common and widely used choice is a mixture of Gaussians:

$$b_j(\mathbf{o}) = \sum_{m=1}^{M} c_{jm} \, \mathcal{N}(\mathbf{o}; \boldsymbol{\mu}_{jm}, \boldsymbol{\Sigma}_{jm}), \tag{13}$$

where $c_{jm} \geq 0$ and $\sum_m c_{jm} = 1$ [4]. This formulation is used extensively in speech recognition and time-series analysis. For the special case $M = 1$, the model reduces to a single Gaussian per state, and the Baum-Welch updates have closed-form M-step solutions for the mean and covariance.

IV. Algorithms and Methods

A. The Forward-Backward Algorithm

The forward-backward algorithm (also called the Baum-Welch E-step or the alpha-beta algorithm) computes all posterior marginals $\gamma_t(i)$ and $\xi_t(i,j)$ in $O(N^2 T)$ time and $O(NT)$ space.

Algorithm 1: Forward-Backward.

1) Initialization: For each state $i \in [N]$, set $\alpha_1(i) = \pi_i b_i(o_1)$ and $\beta_T(i) = 1$.
2) Forward pass (induction): For $t = 1, \ldots, T-1$ and each $j \in [N]$:

$$\alpha_{t+1}(j) = b_j(o_{t+1}) \sum_{i=1}^{N} \alpha_t(i) a_{ij}. \tag{14}$$

3) Backward pass (induction): For $t = T-1, \ldots, 1$ and each $i \in [N]$:

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(o_{t+1}) \beta_{t+1}(j). \tag{15}$$

4) Compute posterior marginals using (11) and (12).
5) Compute likelihood: $P(\mathbf{o} \mid \lambda) = \sum_{i=1}^{N} \alpha_T(i)$.

Numerical Stability. Floating-point underflow is a practical concern when T is large, as $\alpha_t(i)$ can become arbitrarily small. The standard remedy is log-domain computation using the log-sum-exp trick, or equivalently, scaling: at each time step t one divides the forward variables by $c_t = \sum_i \alpha_t(i)$ and accumulates $\log c_t$ separately. The log-likelihood is then $\sum_{t=1}^{T} \log c_t$ [4].

B. The Viterbi Algorithm

The Viterbi algorithm [5] solves Problem 2 (decoding) by finding the MAP state sequence via dynamic programming on the lattice of possible state trajectories.

Define the best-path probability:

$$\delta_t(i) = \max_{s_{1:t-1}} P(S_1 = s_1, \ldots, S_{t-1} = s_{t-1}, S_t = i, O_1 = o_1, \ldots, O_t = o_t \tag{16}$$

Algorithm 2: Viterbi Decoding.

1) Initialization: $\delta_1(i) = \pi_i b_i(o_1)$; $\psi_1(i) = 0$ for all i.
2) Induction: For $t = 2, \ldots, T$ and each $j \in [N]$:

$$\delta_t(j) = \max_{i \in [N]} \left[\delta_{t-1}(i) \, a_{ij} \right] b_j(o_t), \tag{17}$$

$$\psi_t(j) = \arg\max_{i \in [N]} \left[\delta_{t-1}(i) \, a_{ij} \right]. \tag{18}$$

3) Termination: $P^* = \max_i \delta_T(i)$; $s_T^* = \arg\max_i \delta_T(i)$.
4) Path backtracking: For $t = T-1, \ldots, 1$: $s_t^* = \psi_{t+1}(s_{t+1}^*)$.

The Viterbi algorithm runs in $O(N^2 T)$ time. It returns the single most probable path, which may differ from the sequence obtained by independently picking the most probable state at each time step via γ_t.

Remark 1. The Viterbi algorithm is equivalent to max-product belief propagation on the chain graphical model. The forward-backward algorithm is equivalent to sum-product belief propagation on the same model.

C. The Baum-Welch Algorithm

The Baum-Welch algorithm [1] is an instance of EM tailored to HMMs. It iteratively maximizes the expected complete-data log-likelihood.

Algorithm 3: Baum-Welch.

1) Initialize: Choose $\lambda^{(0)} = (\mathbf{A}^{(0)}, \mathbf{B}^{(0)}, \boldsymbol{\pi}^{(0)})$, e.g., by random perturbation of uniform parameters.
2) E-step: Given $\lambda^{(k)}$, run the forward-backward algorithm to compute $\gamma_t^{(k)}(i)$ and $\xi_t^{(k)}(i,j)$ for all t, i, j.
3) M-step: Compute updated parameters:

$$\hat{\pi}_i = \gamma_1(i), \tag{19}$$

$$\hat{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \tag{20}$$

$$\hat{b}_j(k) = \frac{\sum_{t=1}^{T} \gamma_t(j)\, \mathbf{1}[o_t = v_k]}{\sum_{t=1}^{T} \gamma_t(j)}. \tag{21}$$

4) Convergence check: If $|P(\mathbf{o} \mid \lambda^{(k+1)}) - P(\mathbf{o} \mid \lambda^{(k)})| < \epsilon$, stop; otherwise set $k \leftarrow k+1$ and go to step 2.

Equations (19)–(21) have natural interpretations: $\hat{\pi}_i$ is the expected occupancy of state i at time 1; $\hat{a}_{ij}$ is the expected number of transitions from i to j divided by the expected number of transitions leaving i; $\hat{b}_j(k)$ is the expected number of times symbol v_k is emitted in state j divided by the expected number of times the chain occupies state j.

The Baum-Welch algorithm is guaranteed to converge to a local maximum of the likelihood surface [6]. Multiple random restarts are recommended in practice to mitigate sensitivity to initialization.

D. Model Selection: Choosing the Number of States

The number of hidden states N is a hyperparameter that must be selected. Common criteria include:

- BIC: $\mathrm{BIC}(\lambda) = -2 \log P(\mathbf{o} \mid \hat{\lambda}) + |\boldsymbol{\theta}| \log T$, where $|\boldsymbol{\theta}|$ is the number of free parameters.
- AIC: $\mathrm{AIC}(\lambda) = -2 \log P(\mathbf{o} \mid \hat{\lambda}) + 2|\boldsymbol{\theta}|$.
- Cross-validation on held-out sequences.
- Nonparametric Bayesian approaches such as the HDP-HMM [19], which infer N from data.

V. Worked Examples

A. The Dishonest Casino

The dishonest casino is a canonical pedagogical example [10]. A casino uses two dice: a fair die (state F) and a loaded die (state L). The casino switches between dice according to a Markov chain and the player observes only the outcomes.

Model parameters are:

$$\boldsymbol{\pi} = (0.5,\, 0.5),$$

$$\mathbf{A} = \begin{pmatrix} 0.95 & 0.05 \\ 0.10 & 0.90 \end{pmatrix},$$

$$b_F(k) = 1/6 \text{ for } k = 1, \ldots, 6,$$

$$b_L(k) = \begin{cases} 0.1 & k \in \{1,2,3,4,5\} \\ 0.5 & k = 6 \end{cases}.$$

Consider the observation sequence $\mathbf{o} = (3, 6, 6, 1, 6)$ of length $T = 5$.

Forward computation. Initializing with $\alpha_1(F) = 0.5 \times (1/6) = 0.0833$ and $\alpha_1(L) = 0.5 \times 0.1 = 0.05$.

At $t = 2$, $o_2 = 6$:

$$\begin{aligned} \alpha_2(F) &= [\alpha_1(F) a_{FF} + \alpha_1(L) a_{LF}] \cdot b_F(6) \\ &= [0.0833 \times 0.95 + 0.05 \times 0.10] \times \tfrac{1}{6} \\ &= [0.07914 + 0.005] \times 0.1667 \approx 0.01402, \\ \alpha_2(L) &= [\alpha_1(F) a_{FL} + \alpha_1(L) a_{LL}] \cdot b_L(6) \\ &= [0.0833 \times 0.05 + 0.05 \times 0.90] \times 0.5 \\ &= [0.004165 + 0.045] \times 0.5 \approx 0.02458. \end{aligned}$$

Continuing this recursion through $t = 5$ yields the total likelihood $P(\mathbf{o} \mid \lambda) = \alpha_5(F) + \alpha_5(L)$.

Viterbi decoding. At $t = 1$: $\delta_1(F) = 0.0833$, $\delta_1(L) = 0.05$.

At $t = 2$, $o_2 = 6$:

$$\delta_2(F) = \max(0.0833 \times 0.95,\, 0.05 \times 0.10) \times \tfrac{1}{6} = 0.07914 \times 0.1667 \approx 0.$$

$$\delta_2(L) = \max(0.0833 \times 0.05,\, 0.05 \times 0.90) \times 0.5 = 0.045 \times 0.5 = 0.0225$$

The backpointers at $t = 2$ are $\psi_2(F) = F$ and $\psi_2(L) = L$, indicating that at $t = 2$ both states are most likely reached from the same state, consistent with the high self-transition probabilities. For the observation sequence with multiple sixes, the Viterbi path typically identifies a segment attributed to state L (loaded die), demonstrating HMM's ability to segment sequences by latent structure.

B. Numerical Demonstration: Two-State Weather Model

Table I gives a two-state weather HMM modeling "Sunny" (S_1) and "Rainy" (S_2) states with discrete observations from the set {Walk, Shop, Clean} (denoted W, S, C).

TABLE I
Two-State Weather HMM Parameters

Parameter	Sunny (S_1)	Rainy (S_2)
π_i	0.6	0.4
$a_{i,\text{Sunny}}$	0.7	0.4
$a_{i,\text{Rainy}}$	0.3	0.6
$b_i(W)$	0.5	0.1
$b_i(S)$	0.4	0.3
$b_i(C)$	0.1	0.6

For the observation sequence $\mathbf{o} = (W, C, S)$, the forward algorithm yields:

$t = 1$, $o_1 = W$: $\alpha_1(1) = 0.6 \times 0.5 = 0.300$, $\alpha_1(2) = 0.4 \times 0.1 = 0.040$.

$t = 2$, $o_2 = C$:

$$\alpha_2(1) = [0.300 \times 0.7 + 0.040 \times 0.4] \times 0.1 = [0.210 + 0.016] \times 0.1 = 0.0226,$$
$$\alpha_2(2) = [0.300 \times 0.3 + 0.040 \times 0.6] \times 0.6 = [0.090 + 0.024] \times 0.6 = 0.0684.$$

$t = 3$, $o_3 = S$:

$$\alpha_3(1) = [0.0226 \times 0.7 + 0.0684 \times 0.4] \times 0.4 = [0.01582 + 0.02736] \times 0.4 = 0.01727,$$
$$\alpha_3(2) = [0.0226 \times 0.3 + 0.0684 \times 0.6] \times 0.3 = [0.00678 + 0.04104] \times 0.3 = 0.01434.$$

Total likelihood: $P(\mathbf{o} \mid \lambda) = 0.01727 + 0.01434 = 0.03161$.

The posterior at $t = 3$ is $\gamma_3(1) = 0.01727/0.03161 \approx 0.546$ and $\gamma_3(2) \approx 0.454$, suggesting the final observation is slightly more consistent with a Sunny underlying state.

C. Complexity Summary

Table II summarizes the time and space complexity of the three core HMM algorithms.

TABLE II
Complexity of Core HMM Algorithms

Algorithm	Solves	Time	Space
Forward	Evaluation	$O(N^2T)$	$O(NT)$
Forward-Backward	Posterior decoding	$O(N^2T)$	$O(NT)$
Viterbi	MAP decoding	$O(N^2T)$	$O(NT)$
Baum-Welch (per iter.)	Learning	$O(N^2T)$	$O(NT)$

VI. Applications in Data Science

A. Speech Recognition

The application of HMMs to automatic speech recognition (ASR) was transformative [3], [4]. In the classical HMM-based ASR pipeline, each phoneme is modeled by an HMM with three to five emitting states, and continuous-density Gaussian mixture emissions model the acoustic feature vectors (typically mel-frequency cepstral coefficients). Word models are formed by concatenating phoneme HMMs, and sentence-level decoding is performed via the Viterbi algorithm on the resulting composite lattice. Despite the rise of end-to-end deep learning systems, HMM-based hybrid architectures (combining deep neural network acoustic models with HMM topology) remain competitive and provide interpretable alignments [23]. The HMM framework naturally handles the variable-length nature of spoken words and captures the sequential dependency of acoustic features.

B. Bioinformatics: Gene Finding and Protein Modeling

HMMs are a cornerstone tool in computational biology [8], [10]. Profile HMMs model protein sequence families: each position in a multiple sequence alignment is represented by a match state with emission probabilities derived from the column's amino acid composition, flanked by insert and delete states. The HMMER software suite [11], widely used for protein homology search, is built on this foundation. In gene finding, HMMs and generalizations thereof (e.g., generalized HMMs or semi-HMMs [10]) model the statistical properties of coding regions, splice sites, and intergenic regions, enabling probabilistic annotation of genomic sequences.

C. Financial Econometrics

Hamilton [7] introduced regime-switching models, which are HMMs applied to macroeconomic time series. In this framework, a latent state variable captures the current state of the business cycle (e.g., expansion vs. recession), with different Gaussian emission distributions for GDP growth in each regime. This model captures asymmetric behavior over the business cycle and provides a probabilistic recession indicator. Extensions include Markov-switching GARCH models for volatility clustering [24] and applications to algorithmic trading, where HMMs detect market microstructure regimes. The HMM provides forecasting probabilities for regime membership at future horizons via iterated application of the transition matrix to the current posterior state distribution.

D. Natural Language Processing

Before the widespread adoption of neural sequence models, HMMs were the dominant tool for many NLP tasks. Part-of-speech (POS) tagging models the tag sequence as the latent state chain and the word sequence as the observations [22]. Named entity recognition and chunking were similarly cast as HMM inference problems. The first-order HMM corresponds to a bigram tag language model; higher-order HMMs capture richer tag context. While modern NLP relies heavily on conditional random fields [12] and transformer architectures, HMMs remain valuable for low-resource settings and for interpretable models where the latent segmentation is of interest.

E. Activity Recognition and Gesture Modeling

HMMs are widely applied to recognize human activities and gestures from sensor data [9]. In gesture recognition, each gesture class is modeled by a left-to-right HMM (an HMM with an upper triangular transition matrix, reflecting temporal progression). Recognition is performed via likelihood comparison across class-specific HMMs. In wearable sensor applications, the latent states capture postures or movement phases, and the emissions model accelerometer or gyroscope readings. This application illustrates the HMM's ability to handle structured temporal dependencies in complex multivariate sensor data.

F. Anomaly Detection

A natural application of HMMs is anomaly detection in sequential data [25]. After training an HMM on normal behavior, the log-likelihood of a new sequence under the model serves as an anomaly score: sequences with low likelihood are flagged as anomalous. This approach has been applied to intrusion detection in network logs, fault

detection in industrial machinery, and fraud detection in transaction sequences. The posterior state sequence from the Viterbi algorithm can further localize the anomalous segment.

VII. Limitations and Assumptions

While HMMs are powerful and well-understood, their utility depends on the validity of several modeling assumptions that may not hold in all settings.

First-order Markov assumption. The standard HMM assumes that the current state depends only on the immediately preceding state. Many real processes exhibit longer-range dependencies. Higher-order HMMs or duration models can partially address this, at the cost of exponentially growing parameter spaces.

Conditional independence of observations. Equation (3) asserts that observations are independent given the hidden states. In practice, observations often exhibit residual correlations beyond what the state structure captures. Autoregressive-HMMs, which model each emission as a function of both the current state and previous observations, provide a generalization [16].

Discrete, time-homogeneous transitions. Standard HMMs assume transition probabilities are constant over time. Non-stationary processes, such as seasonal time series or systems undergoing regime change, may require time-varying transition matrices or explicit covariate-driven transitions.

Local optima in Baum-Welch. The EM algorithm converges only to a local maximum of the likelihood, and the HMM likelihood surface is typically multimodal. The learned solution is sensitive to initialization. Spectral methods [20] provide globally convergent alternatives under identifiability conditions, though they are less widely used in practice.

Model identifiability. HMMs are identifiable up to label permutation of the hidden states under mild conditions [21]. However, in practice, with small T or nearly identical emission distributions, the states can be difficult to disentangle.

Computational scaling. While $O(N^2T)$ is polynomial and fast for moderate N, very large state spaces (e.g., $N > 10^3$) make exact inference expensive. Approximate inference methods, including variational approaches and particle filtering, are required in such settings.

VIII. Connections to Related Methods

A. Kalman Filter

The Kalman filter [15] is the continuous-state analogue of the HMM. It assumes the latent state $\mathbf{s}_t \in \mathbb{R}^d$ evolves according to a linear Gaussian transition $\mathbf{s}_t = \mathbf{A}\mathbf{s}_{t-1} + \boldsymbol{\epsilon}_t$ and produces observations $\mathbf{o}_t = \mathbf{C}\mathbf{s}_t + \boldsymbol{\eta}_t$ with Gaussian noise. The forward algorithm for the Kalman filter reduces to the Kalman prediction and update equations, and the Kalman smoother is the continuous analogue of the forward-backward algorithm. The linear-Gaussian structure enables exact, closed-form inference, in contrast to the finite-state summations of the HMM. Switching state-space models [18] combine discrete Markov switching (as in HMMs) with continuous Kalman dynamics, providing a richer model at the cost of approximate inference.

B. Conditional Random Fields

Conditional Random Fields (CRFs) [12] are discriminative counterparts to HMMs. A linear-chain CRF directly models the conditional distribution $P(\mathbf{s}_{1:T} \mid \mathbf{o}_{1:T})$ via a log-linear feature function:

$$P(\mathbf{s} \mid \mathbf{o}) \propto \exp\left(\sum_t \sum_k \theta_k f_k(s_t, s_{t-1}, \mathbf{o}, t)\right), \qquad (22)$$

without requiring a generative model for the observations. CRFs avoid the label bias problem of maximum entropy Markov models and typically achieve higher accuracy on discriminative tasks such as POS tagging and named entity recognition [12]. However, they lack the generative semantics needed for tasks such as sequence synthesis or anomaly scoring.

C. Recurrent Neural Networks and LSTMs

Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs) [13] can be viewed as parameterizing the same class of sequential dependencies as HMMs, but with continuous, distributed hidden states and non-linear dynamics learned via backpropagation through time. While RNN/LSTMs are more expressive, they sacrifice interpretability and the principled probabilistic uncertainty quantification inherent to HMMs. Hybrid architectures, such as the DNN-HMM used in modern ASR [23], combine the representational power of neural networks with the structured probabilistic framework of HMMs.

D. Latent Dirichlet Allocation

Latent Dirichlet Allocation (LDA) [14] is a bag-of-words topic model that, like HMMs, discovers latent categorical structure in complex data. Unlike HMMs, LDA does not model sequential order; it assumes documents are exchangeable mixtures of topics. The Hidden Markov model can be seen as a sequential version of mixture modeling, with the crucial difference that the latent assignment at each position is Markov-dependent on adjacent positions rather than i.i.d.

E. Factorial and Hierarchical HMMs

Factorial HMMs [16] extend the basic model to multiple independent chains of latent variables whose emissions combine to produce the observation. This allows richer representations without an exponential blowup in state space (at the cost of approximate inference). Hierarchical HMMs [17] introduce a multi-level Markov structure where

higher-level states govern the dynamics of lower-level sub-HMMs, enabling modeling of hierarchically structured sequences such as sentences, paragraphs, and documents.

IX. Conclusion

This survey has provided a comprehensive treatment of Hidden Markov Models, covering their mathematical foundations, the three canonical problems of evaluation, decoding, and learning, the corresponding forward-backward, Viterbi, and Baum-Welch algorithms, worked numerical examples, and a broad review of applications across data science.

HMMs occupy a central position in the modeling of dependent sequential data. Their explicit latent structure provides a principled mechanism for identifying hidden regimes and segmenting sequences—directly addressing the data science theme of discovering latent structure in complex data. The Markov chain dynamics encode structured temporal dependencies, addressing the theme of identifying and modeling dependencies in sequential observations. Forecasting under an HMM is achieved by propagating the posterior state distribution through the transition matrix, naturally accounting for the dependence structure. The unsupervised Baum-Welch learning procedure extracts latent organization from unlabeled sequences.

Despite the growing prevalence of deep sequential models, HMMs remain relevant for their interpretability, their exact inference algorithms, their principled probabilistic uncertainty quantification, and their computational efficiency for moderate state spaces. Modern advances—including nonparametric Bayesian HMMs, spectral learning methods, and hybrid neural-HMM architectures—continue to extend their applicability. For the data scientist and statistician, HMMs provide a foundational yet powerful toolkit for reasoning about the hidden mechanisms that generate observable sequential data.

References

[1] L. E. Baum, T. Petrie, G. Soules, and N. Weiss, "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains," The Annals of Mathematical Statistics, vol. 41, no. 1, pp. 164–171, 1970.

[2] J. K. Baker, "The DRAGON system—an overview," IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 23, no. 1, pp. 24–29, 1975.

[3] F. Jelinek, L. R. Bahl, and R. L. Mercer, "Design of a linguistic statistical decoder for the recognition of continuous speech," IEEE Transactions on Information Theory, vol. 21, no. 3, pp. 250–256, 1975.

[4] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257–286, 1989.

[5] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE Transactions on Information Theory, vol. 13, no. 2, pp. 260–269, 1967.

[6] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," Journal of the Royal Statistical Society: Series B, vol. 39, no. 1, pp. 1–22, 1977.

[7] J. D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," Econometrica, vol. 57, no. 2, pp. 357–384, 1989.

[8] A. Krogh, M. Brown, I. S. Mian, K. Sjolander, and D. Haussler, "Hidden Markov models in computational biology: Applications to protein modeling," Journal of Molecular Biology, vol. 235, no. 5, pp. 1501–1531, 1994.

[9] J. Yamato, J. Ohya, and K. Ishii, "Recognizing human action in time-sequential images using hidden Markov model," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1992, pp. 379–385.

[10] R. Durbin, S. R. Eddy, A. Krogh, and G. Mitchison, Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids. Cambridge, U.K.: Cambridge University Press, 1998.

[11] S. R. Eddy, "Profile hidden Markov models," Bioinformatics, vol. 14, no. 9, pp. 755–763, 1998.

[12] J. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in Proceedings of the 18th International Conference on Machine Learning (ICML), 2001, pp. 282–289.

[13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[14] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," Journal of Machine Learning Research, vol. 3, pp. 993–1022, 2003.

[15] R. E. Kalman, "A new approach to linear filtering and prediction problems," Journal of Basic Engineering, vol. 82, no. 1, pp. 35–45, 1960.

[16] Z. Ghahramani and M. I. Jordan, "Factorial hidden Markov models," in Advances in Neural Information Processing Systems (NIPS), 1996, pp. 472–478.

[17] S. Fine, Y. Singer, and N. Tishby, "The hierarchical hidden Markov model: Analysis and applications," Machine Learning, vol. 32, no. 1, pp. 41–62, 1998.

[18] C.-J. Kim, "Dynamic linear models with Markov-switching," Journal of Econometrics, vol. 60, no. 1-2, pp. 1–22, 1994.

[19] Y. W. Teh, M. I. Jordan, M. J. Beal, and D. M. Blei, "Hierarchical Dirichlet processes," Journal of the American Statistical Association, vol. 101, no. 476, pp. 1566–1581, 2006.

[20] D. Hsu, S. M. Kakade, and T. Zhang, "A spectral algorithm for learning hidden Markov models," Journal of Computer and System Sciences, vol. 78, no. 5, pp. 1460–1480, 2012.

[21] E. S. Allman, C. Matias, and J. A. Rhodes, "Identifiability of parameters in latent structure models with many observed variables," The Annals of Statistics, vol. 37, no. 6A, pp. 3099–3132, 2009.

[22] J. Kupiec, "Robust part-of-speech tagging using a hidden Markov model," Computer Speech & Language, vol. 6, no. 3, pp. 225–242, 1992.

[23] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in Proceedings of the 23rd International Conference on Machine Learning (ICML), 2006, pp. 369–376.

[24] J. D. Hamilton, "Regime-switching models," in New Palgrave Dictionary of Economics, S. Durlauf and L. Blume, Eds. Basingstoke, U.K.: Palgrave McMillan, 2005.

[25] T. Lane and C. E. Brodley, "Temporal sequence learning and data reduction for anomaly detection," ACM Transactions on Information and System Security, vol. 2, no. 3, pp. 295–331, 1999.